

Employee Attrition Prediction Project

Goal

The objective was to predict whether an employee would leave the company.

Target variable:

left_company

0 -> Employee stays

1 -> Employee leaves

This is a **binary classification problem**, making Random Forest a good initial choice.

Phase 1: Understanding the Dataset

The dataset contained employee-related features such as

- Age
- Salary
- Years at company
- Job Satisfaction
- Work Life Balance
- Overtime
- Department
- Performance Rating
- etc.

One column

employee_id

was removed because it uniquely identifies employees but contains no predictive information. Our initial features became

```
X = emp.drop(["employee_id", "left_company"], axis=1)
```

Target

```
y = emp["left_company"]
```

Phase 2: First Random Forest

We trained

```
RandomForestClassifier(  
    n_estimators=100  
)
```

using

```
train_test_split()
```

with

```
test_size=0.5
```

Later we improved reproducibility by adding

```
random_state=42
```

and

```
stratify=y
```

Why stratify?

Without stratification

each split is random.

You could accidentally get

Training

80% stay

20% leave

Testing

60% stay

40% leave

which makes evaluation unfair.

Using

stratify=y

keeps the same class proportions in both sets.

Phase 3: First Evaluation

We learned that **accuracy alone is misleading**.

Instead we evaluated

- Accuracy
 - Precision
 - Recall
 - F1-score
 - Confusion Matrix
-

Our first model produced roughly

Accuracy $\approx$ 80%

which initially looked decent.

But then we inspected

Recall for class 1

and discovered

Recall $\approx$ 0.28

meaning

Most employees who resigned
were never detected.

This was the first major lesson.

Phase 4: Understanding the Confusion Matrix

Instead of trusting accuracy

we started reading

TP

TN

FP

FN

For HR

False Negatives

Employee leaves

↓

Model predicts stay

are expensive.

The company loses employees unexpectedly.

This shifted our focus from

Highest Accuracy

to

Best Recall + Best F1

Phase 5: Detecting Overfitting

We compared

Training Accuracy

vs

Testing Accuracy

Initially

Train = 100%

Test $\approx$ 80%

which immediately suggested

Overfitting

The trees were memorizing the training data.

Phase 6: Hyperparameter Tuning

Instead of randomly changing everything

we changed **one parameter at a time**.

This is exactly how controlled experiments should be performed.

Experiment 1

n_estimators

Question

Do more trees improve the forest?

We tested

10

20

30

...

200

Result

Accuracy improved slightly

then plateaued.

Around

100 trees

the gains became tiny.

Lesson

More trees

↓

Better stability

↓

Diminishing returns

Experiment 2

max_depth

Question

How deep should each tree grow?

We tested

1

2

3

...

20

Observation

Training accuracy rose quickly

Eventually

Train = 100%

Testing accuracy peaked around

Depth ≈ 7

Beyond that

the trees memorized the training data.

We also tracked

Recall

F1

instead of only accuracy.

This showed

Depth ≈ 7

gave the best balance.

Experiment 3

class_weight="balanced"

This became the biggest conceptual lesson.

Your data was roughly

76%

Stay

24%

Leave

Without balancing

the forest naturally learns

Predict stay

most of the time

because that's the easiest way to maximize accuracy.

Adding

`class_weight="balanced"`

tells Random Forest

Missing someone who leaves

↓

Should cost more.

Immediately

Recall jumped dramatically.

This showed

Business objective

>

Raw Accuracy

Experiment 4

max_depth + class_weight

We combined

Balanced

+

Depth tuning

The curves became smoother.

The model generalized better.

Experiment 5

min_samples_leaf

Question

How small should a leaf be allowed to become?

Instead of allowing tiny leaves

we forced

each leaf to contain at least

1

2

3

...

20 samples

This turned out to be one of the most useful parameters.

Training accuracy gradually decreased

while

Recall

and

F1

improved.

The sweet spot was around

15

This parameter greatly reduced overfitting.

Experiment 6

min_samples_split

Question

How many samples are required

before a node may split?

Surprisingly

almost nothing changed.

The graphs looked almost scripted.

That taught an important lesson.

Sometimes
a hyperparameter simply
doesn't matter
because another one
is already controlling the tree.

In our case
`min_samples_leaf`

was already acting as the dominant constraint.

Phase 7

Department Feature

Initially

we removed

department

because it was categorical.

We later learned

Random Forest cannot understand text.

So we introduced

OneHotEncoder

using

ColumnTransformer

This converted

Engineering
HR

Sales

...

into binary columns.

The feature count increased

from

12

↓

17

Phase 8

Comparing Department

We kept

every hyperparameter fixed

and changed only

one thing

Department

added

vs

removed

This made the comparison fair.

The tuned model with

Department

produced

approximately

Recall

0.77

F1

0.65

instead of

Recall

0.28

F1

0.41

That is a huge improvement.

The confusion matrix changed dramatically.

Without department

Detected

12

employees leaving

Missed

31

With department

Detected

33

Missed

10

Accuracy stayed roughly similar

but

business usefulness

increased enormously.

Phase 9

Removing Balanced

We then removed

`class_weight="balanced"`

while keeping department.

The result

Recall

0.05

Precision

1.00

Initially

Precision looked amazing.

But then we interpreted the confusion matrix.

The model predicted

almost everyone

would stay.

It only predicted

2 employees

would leave.

Both happened to be correct.

Hence

Precision = 100%

but

Recall = 5%

This was essentially

a useless HR model.

Biggest Lessons

Accuracy is not enough.

Always inspect

- Precision
 - Recall
 - F1
 - Confusion Matrix
-

Business objective matters.

For HR

missing resignations

is worse

than interviewing

a few extra employees.

That makes
Recall
more valuable than
raw accuracy.

Hyperparameter tuning should be systematic.

Instead of changing
five things
at once
change
one
parameter
while fixing everything else.
That allows you to understand
cause and effect.

Some hyperparameters matter much more than others.

For your dataset

roughly
class_weight
★★★★★
department
★★★★★
min_samples_leaf
★★★★☆

max_depth

★★★★☆

n_estimators

★★☆☆☆

min_samples_split

★☆☆☆☆

More complex isn't always better.

The highest training accuracy

was often

100%

but those models

generalized poorly.

Feature engineering can outperform hyperparameter tuning.

Adding

department

had a larger impact

than several tuning experiments combined.

That is a very common theme in machine learning.

Final Best Model

Your strongest model was approximately:

```
RandomForestClassifier(  
    n_estimators=100,  
    random_state=42,  
    class_weight="balanced",  
    max_depth=7,  
    min_samples_leaf=15  
)
```

with the **department** feature included through **OneHotEncoding**.

What You Actually Learned

Although this started as "learn Random Forest," you ended up practicing the complete workflow of a machine learning practitioner:

1. Understanding the dataset and defining the target.
2. Preparing features and handling categorical variables.
3. Creating reproducible train/test splits with stratification.
4. Training a baseline model.
5. Evaluating beyond accuracy using precision, recall, F1, and the confusion matrix.
6. Diagnosing overfitting by comparing training and testing performance.
7. Systematically tuning hyperparameters one at a time.
8. Interpreting why some hyperparameters mattered and others did not.
9. Comparing feature sets through controlled experiments.
10. Choosing the best model based on the problem's business objective rather than the highest accuracy.

That's a complete end-to-end supervised machine learning project, and it mirrors the thought process used in many real-world classification tasks.